

Aula 02/15

Escola Tecnológica FPFtech

Curso Técnico: Informática para Internet

Módulo: Testes de Software

Prof. Denis Moraes Guimarães

22 de maio de 2026

Sumário

- ▶ Tipos de teste
- ▶ Níveis de teste
- ▶ Técnicas de teste
- ▶ Estudo de caso: Calculadora

Tipos de teste

Quanto ao objetivo:

- ▶ Funcional
- ▶ Não funcional

Quanto à quantidade de informação disponível:

- ▶ Teste de caixa preta (black-box)
- ▶ Teste de caixa branca (white-box)
- ▶ Teste de caixa cinza (gray-box)

Tipos de teste - Funcionais

- ▶ Avalia as funções que um componente ou sistema deve executar.
- ▶ As funções são "o que" o objeto de teste deve fazer.
- ▶ O principal objetivo do teste funcional é verificar a integridade funcional, a correção funcional e a adequação funcional.



Tipos de teste - Não funcionais

- ▶ Avalia atributos que não sejam características funcionais de um componente ou sistema.
- ▶ O teste não funcional é o teste de "quão bem o sistema se comporta".
- ▶ O principal objetivo do teste não funcional é verificar as características não funcionais da qualidade do software.



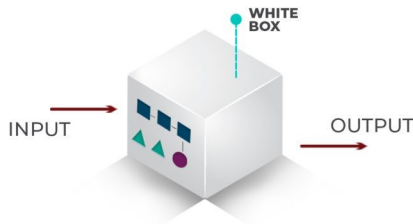
Tipos de teste - Caixa Preta

- ▶ É baseado em especificações e deriva testes da documentação externa ao objeto de teste.
- ▶ O principal objetivo do teste caixa-preta é verificar o comportamento do sistema em relação às suas especificações.



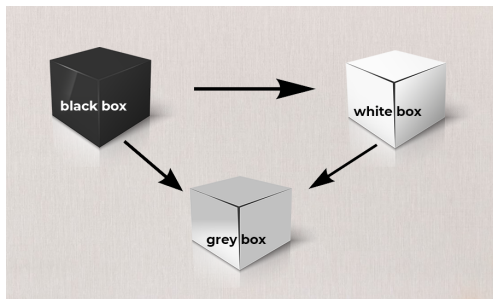
Tipos de teste - Caixa Branca

- ▶ É baseado na estrutura e deriva testes da implementação ou da estrutura interna do sistema (p. ex., código, arquitetura, fluxos de trabalho e fluxos de dados).
- ▶ O principal objetivo do teste caixa-branca é cobrir a estrutura subjacente pelos testes até o nível aceitável.



Tipos de teste - Caixa Cinza

- ▶ O testador possui conhecimento parcial da estrutura interna do sistema.
- ▶ Parte da arquitetura, banco de dados, APIs, fluxo interno, diagramas, estrutura de dados, regras técnicas.



Níveis de teste

- ▶ Teste de componente
- ▶ Teste de integração de componentes
- ▶ Teste de sistema
- ▶ Teste de integração de sistemas
- ▶ Testes de aceite

Níveis de teste - Teste de componentes (unitário)

- ▶ Concentra-se em testar componentes isoladamente.
- ▶ Geralmente, requer suporte específico, como estruturas de teste ou frameworks de teste de unidade
- ▶ Normalmente, são realizados por desenvolvedores em seus ambientes de desenvolvimento.

```
>>> assert 1+1 == 2
>>> assert 1+1 == 3, "The sum should be 2"
Traceback (most recent call last):
  File "<input>", line 1, in <module>
AssertionError: The sum should be 2
```

Níveis de teste - Teste de integração de componentes

- ▶ Teste das interfaces e interações entre os componentes.
- ▶ Depende muito das abordagens da estratégia de integração, como bottom-up, top-down ou big-bang.
- ▶ Normalmente, são realizados por desenvolvedores em seus ambientes de desenvolvimento.
- ▶ É desenvolvido driver ou stub para simular o comportamento de componentes que ainda não foram integrados.

```
from unittest import mock

@mock.patch("dnd.randint", return_value=5, autospec=True)
def test_attack_damage(mock_randint):
    assert attack_damage(1) == 6
    mock_randint.assert_called_once_with(1, 8)
```

Níveis de teste - Teste de sistema

- ▶ Comportamento geral e nos recursos de todo um sistema ou produto.
- ▶ Geralmente incluindo o teste funcional de tarefas de ponta a ponta e o teste não funcional de características de qualidade.



Níveis de teste - Teste de integração de sistemas

- ▶ Concentra-se no teste das interfaces do sistema e de outros sistemas e serviços externos.
- ▶ Requer ambientes de teste adequados, de preferência semelhantes ao ambiente operacional.



Níveis de teste - Testes de aceite

- ▶ Validação e na demonstração da disposição para a implantação.
- ▶ Sistema atende às necessidades (requisitos) de negócio do usuário.
- ▶ Deve ser realizado pelos usuários previstos.



Técnicas de teste

Caixa Preta:

- ▶ Particionamento de equivalência
- ▶ Análise de valor limite
- ▶ Teste de tabela de decisão
- ▶ Teste de transição do estado

Caixa Branca:

- ▶ Teste de instruções
- ▶ Teste de ramificação

Técnicas de teste - Particionamento de equivalência

- ▶ Reduzir a quantidade de casos de teste sem perder cobertura importante.
- ▶ Imagine um campo de idade, de 18 até 60
- ▶ Muitos casos de teste possíveis, mas podemos dividir em classes de equivalência.
- ▶ Exemplo: Campo idade - Partição válida: 18-60; Teste: 30.
- ▶ Partição inválida: $x < 18$; Teste: 15.
- ▶ Partição inválida: $x > 60$; Teste: 65.

Técnicas de teste - Análise de Valor Limite

- ▶ Encontrar defeitos nos limites das entradas.
- ▶ Muitos erros acontecem exatamente nas bordas dos intervalos válidos.
- ▶ Testa o vizinho da partição adjacente.
- ▶ Exemplo: Campo de idade - 18-60.
- ▶ Limite inferior: 17, 18.
- ▶ Limite superior: 60, 61.

Técnicas de teste - Teste de tabela de decisão

- ▶ Diferentes combinações de condições produzem diferentes ações/resultados.
- ▶ Muito útil quando o sistema possui muitas regras, múltiplas condições.
- ▶ Exemplo: Login - usuário só acessa se senha correta e conta ativa.

Regra	Senha correta	Conta ativa	Permitir acesso
R1	V	V	X
R2	V	F	
R3	F	V	
R4	F	F	

Técnicas de teste - Análise de Valor Limite

- ▶ O comportamento do sistema depende do estado atual em que ele está.
- ▶ O sistema pode responder de forma diferente ao mesmo evento dependendo do estado atual.
- ▶ Exemplo: Conta bancária
- ▶ Transição inválida: Logout estando deslogado, Comprar sem login.

Estado Atual	Evento	Próximo Estado
Deslogado	Login correto	Logado
Deslogado	3 erros	Bloqueado
Logado	Logout	Deslogado

Técnicas de teste - Teste de instruções

- ▶ Verifica se todas as instruções executáveis do código foram executadas pelo menos uma vez.
- ▶ Garantir que cada linha executável do código seja exercitada durante os testes.
- ▶ 100% de cobertura

```
1 def dividir(a, b):  
2     if b != 0:  
3         return a / b  
4  
5 def test_case_1():  
6     assert dividir(10, 2) == 5  
7     assert dividir(20, 0) == 20
```

Técnicas de teste - Teste de instruções

- ▶ Verifica se todos os caminhos de decisão do código foram executados.
- ▶ Garantir que cada possível desvio do fluxo do programa seja testado.

```
1 def verificar(x):  
2     if x > 0:  
3         print("positivo")  
4     else:  
5         print("negativo")  
6  
7 def test_case_1():  
8     assert verificar(5) == "positivo"  
9  
10 def test_case_2():  
11     assert verificar(-3) == "negativo"
```

Estudo de caso - Calculadora

